

AULA 10

Aterrissagem do TP2: o sistema sobe com um comando

Etapa 5 · terça, 22/09/2026 · sala SJ205 · Prof. Dácio Moreira de Souza



Um projeto que só você consegue subir não está entregue.

Se o avaliador não consegue abrir, não existe. Se ele não consegue subir, também não.

O TP2 vence sexta, 25/09 — três dias depois desta aula

- G2.5 · YAML + URDF com TF coerente · vence HOJE
- G2.6 · entrega: branch, tag, relatório, Moodle · vence 25/09
- Esta é a última aula antes da entrega
- O que não for resolvido aqui será resolvido sozinho, em casa, na quinta à noite

A aula está ordenada por custo de perda, não por sequência lógica

- 1. Clínica de URDF e TF — serve o G2.5, vence hoje — não pode cair
- 2. Aterrissagem — serve a entrega, vence 25/09 — não pode cair
- 3. Contrato do detector — serve o relatório, 25/09 — não pode cair
- 4. Launch — serve o G3.0, vence 30/09 — volta na Aula 11 se faltar tempo



O corte vem antes do aperto.

É a mesma disciplina da Aula 9 aplicada ao tempo: a régua vem antes da medida. Escolher de antemão o bloco que cede é decisão; descobrir no fim é acidente.

PARTE 1

Clínica de URDF e TF

Bancada aberta sobre o tutorial, publicado desde 08/09

- O G2.5 pede: parametrização em YAML + URDF carregando com TF coerente
- Evidência: frames.pdf, print do RViz2 e o ros2 param dump do seu nó
- Quem chegar sem ter lido o tutorial vai gastar a clínica instalando pacote
- A evidência do gate sai de dentro da própria clínica — traga o notebook ligado

Comece pelo diagnóstico: ele diz em que ponto você parou

```
bash recursos/clinica-urdf.sh
```

Ele percorre a cadeia inteira do gate, para no primeiro ponto que falhou, e imprime qual é o próximo comando. Rode na raiz do seu ros2_ws.

Depois rode o que ele mandar — e traga o que travou

```
check_urdf src/<seuprojeto>_description/urdf/<seu_robo>.urdf  
ros2 launch <seuprojeto>_description ver_robo.launch.py
```

```
# noutro terminal, com o launch RODANDO:
```

```
ros2 run tf2_tools view_frames  
ros2 run tf2_ros tf2_echo base_footprint camera_link
```

Os três tropeços conhecidos, todos já documentados

- TF exige o launch rodando — sem ele o view_frames gera um PDF vazio sem reclamar
- Janelas vazias no WSL2 — bisseção com o turtlesim, depois LIBGL_ALWAYS_SOFTWARE=1
- O <origin> do joint confundido com o do visual — RViz2 parece certo, TF está errada

O G2.5 não depende da janela gráfica.

`check_urdf`, `view_frames` e `tf2_echo` rodam no terminal e provam exatamente o que o gate pede. Se o RViz2 não abre na sua máquina, você ainda entrega hoje.

PARTE 2

Aterrissagem: o que significa entregar

O G2.6 não é "mandar o trabalho" — são quatro coisas verificáveis

- A branch entrega-tp2, no GitHub
- A tag tp2, que congela o ponto exato do histórico
- O relatório respondendo ao que cada gate pediu
- A postagem no Moodle — que é a entrega oficial e é ela que marca a data

Os três comandos que fecham a parte do GitHub

```
git checkout -b entrega-tp2  
git tag tp2  
git push origin entrega-tp2 --tags
```

Confira depois: `git tag --list` tem de mostrar `tp2`. Sem o `--tags`, a tag fica só na sua máquina.

O relatório responde, no mínimo, ao que cada gate pediu

- G2.0 e G2.1 · as interfaces do seu domínio, e por que elas existem
- G2.2 e G2.3 · a action com feedback e cancelamento demonstrados
- G2.4 · a métrica declarada, os números das duas versões, e onde cada uma falha
- G2.5 · a parametrização, o URDF e a árvore de TF

A armadilha clássica: a action que não cancela.

Se o seu servidor não pode ser cancelado no meio, o G2.3 não passou — e o G2.3 é o gate que a correção efetivamente testa. Se isso ainda está aberto, diga agora, não quinta à noite.

Um gate aberto e documentado custa muito menos que um gate aberto e escondido.

É a mesma honestidade que a Aula 9 pediu na hora de relatar onde o detector falha.

PARTE 3

Trocar o detector sem trocar a régua

O avaliar.py não sabe nada sobre como o seu detector funciona

- Ele chama uma função que recebe a imagem e devolve (x, y, w, h) ou None
- Plugar um modelo treinado é escrever outra função com essa assinatura — nada mais
- A régua sobrevive à troca do detector: é por isso que os números continuam comparáveis
- Mesmo princípio do launch, daqui a pouco: o bringup não sabe o que cada nó faz



O contrato fica, o miolo troca.

É o mesmo ponto das interfaces da Aula 6, agora aplicado à percepção.

Sobre YOLO, torch e a regra da disciplina

- O gate diz "YOLO ou equivalente" — e o equivalente importa
- cv2.dnn roda modelos ONNX e Darknet sem instalar torch nem ultralytics
- É o OpenCV do apt que vocês já têm
- Quer o ecossistema completo? Vai num venv (uv venv --system-site-packages), como script solto — nunca dentro de um nó ROS 2

O que se avalia é a métrica e a justificativa, não o modelo.

Um HSV medido honestamente vale mais que um YOLO sem régua. A três dias da entrega, trocar um HSV medido por um YOLO não medido é piorar a nota.

PARTE 4

Launch: um comando sobe o sistema inteiro

Launch vocês usam desde a Aula 2 — o que muda agora é a escala

- Sair de "um launch por exemplo" para um launch que sobe o projeto todo
- É o que o G3.0 do TP3 cobra em 30/09
- Compor: IncludeLaunchDescription traz o subsistema inteiro, com seus argumentos
- Parametrizar: o YAML vem de fora, então o mesmo launch sobe teste e bancada
- Condicionar: IfCondition liga e desliga pedaços

O exemplo da aula sobe um sistema pequeno e completo

```
ros2 launch aula10_bringup bringup.launch.py  
ros2 launch aula10_bringup bringup.launch.py modelo:=false
```

Sem câmera e sem hardware. O `modelo:=false` sobe sem o pacote da Aula 9.



Um launch que sobe sem erro não é um launch que funciona.

Subir é o que o launch faz. Funcionar é outra afirmação — e ela precisa de prova.

As três verificações

```
ros2 launch aula10_bringup bringup.launch.py --show-args # o que dá para configurar
ros2 node list                                           # quem realmente subiu
ros2 param dump /percepcao/detector                    # com que valores
```

A terceira é a que quase ninguém faz. É ela que pega o erro do próximo slide.

O YAML que ninguém lê

```
ros2 launch aula10_bringup bringup.launch.py \  
  params:=../config/sistema-armadilha.yaml  
  
ros2 param get /percepcao/detector taxa_hz  
-> 2.0          # o padrão do código, não o 9.0 do arquivo
```

O arquivo pedia 9.0. O nó subiu com 2.0. Nenhum erro foi impresso em lugar nenhum.

A causa: a chave do YAML não casa com o nome do nó

- detector: significa o nó /detector, na raiz
- Mas o launch empurrou o nó para /percepcao/detector
- Os nomes não casam, e a regra do ROS 2 é ignorar o que não casa — sem avisar
- O launch subiu, os nós subiram, o arquivo foi lido, os valores foram descartados

É a mesma família de falhas que vocês já encontraram

- O `view_frames` que gera PDF vazio quando o launch não está rodando
- A saturação que vira ruído puro quando o valor é quase zero
- O `ros2 action list` que funciona sem `source` — e o `send_goal` que não
- São erros que não se apresentam como erro



A defesa é sempre a mesma: verificar a afirmação, em vez de tratar o silêncio como aprovação.

E, quando dá, automatizar a verificação — para não depender de lembrar.

As duas curas — e o conferidor, que roda sem ROS 2

```
# cura 1:  /percepcao/detector:  
# cura 2:  /**:                <- prefira esta  
  
ros2 node list | python3 conferir-params.py config/sistema.yaml -
```

Prefira o curinga: ele sobrevive à mudança de namespace — e mudar namespace é exatamente o que você vai fazer quando o sistema crescer.

Sobre ordem: nada espera nada

- O supervisor quase sempre nasce antes do detector, avisa que não recebeu dado, e se recupera
- Isso é o comportamento correto, não defeito
- Em ROS 2 o grafo é descoberto, não montado em ordem
- Um nó que só funciona se subir depois de outro quebra no dia em que a máquina estiver lenta
- E esse dia sempre chega: costuma ser o da apresentação



O nó tolera a ausência do outro.

Se você acha que precisa de ordem de subida, quase sempre precisa é de um nó mais tolerante.

Tarefa da semana: fechar o TP2 e entregar até sexta, 25/09

- Não há tarefa nova — o que existe é a entrega
- Branch entrega-tp2, tag tp2 visível em `git tag --list`
- Relatório respondendo aos seis gates, evidências em `docs/evidencias/tp2/`
- A postagem no Moodle
- Gate que não fechou: diga no relatório o que não fechou e por quê

Para a próxima aula, 29/09 — navegação autônoma

- Abre a Etapa 6, e não é assunto novo: é a montagem do que já está no lugar
- O Nav2 é construído sobre ações (Aula 8) e TF coerente (o G2.5 de hoje)
- É por isso que a clínica de hoje importa além do TP2
- O G3.2 do TP3 pede árvore de TF completa e sem warning
- Noventa por cento dos problemas de SLAM e de Nav2 são problemas de TF disfarçados